Kingdom of Cambodia

Nation Religion King

NovelTTS: Automated Multi-Voice Audiobook Generation for Web Novels Using LLM Dialog Attribution and Free Neural TTS Synthesis

By

STUDENT NAME

Register No: _____

KIRIROM INSTITUTE OF TECHNOLOGY

A PROJECT REPORT

Submitted to the

DEPARTMENT OF ENGINEERING

In partial fulfillment of the requirements

for the award of the degree of

BACHELOR OF ENGINEERING

IN

SOFTWARE ENGINEERING

May, 2026

A PROJECT REPORT

On

NovelTTS: Automated Multi-Voice Audiobook Generation for Web Novels Using LLM Dialog Attribution and Free Neural TTS Synthesis

Submitted to Kirirom Institute of Technology in partial fulfillment of the requirement for the award of the degree of

Bachelor of Engineering

In

SOFTWARE ENGINEERING

Submitted By

Student's Name (Register Number)

SOFTWARE ENGINEERING

Kirirom Institute of Technology, Cambodia

_____, 2026

ACKNOWLEDGEMENT

Heartfelt gratitude is extended to all the people who have directly and indirectly contributed to the development of this project and have had great influence in the thoughts, behavior and acts exhibited during the course of study. Special thanks to the internship company for providing a stimulating environment in which to develop and test the NovelTTS system. The guidance and feedback received throughout the development phases have been invaluable.

Student's Name
Register Number

CERTIFICATE OF DECLARATION

I, Name , Register Number , Bachelor of Engineering (Semester-VII) of the KIRIROM INSTITUTE OF TECHNOLOGY hereby declare that the Project Report entitled "NovelTTS: Automated Multi-Voice Audiobook Generation for Web Novels Using LLM Dialog Attribution and Free Neural TTS Synthesis" is an original work and the data provided in the study is authentic to the best of my knowledge. This report has not been submitted to any other Institute for the award of any other degree.

Student's Name
(Register Number)

Place: Kirirom Institute of Technology

Date: _____

_____

This is to certify that above mentioned project report is the bonafide work of _____ carried out from February 2025 to July 2025 and submitted to Kirirom Institute of Technology in partial fulfillment of the award of the Degree of Bachelor of Engineering in Software Engineering.

Approved by:

_____

Project Supervisor

Noted by:

Dr. Dinesh Kumar Karthikeyan

Deputy Head, Software Engineering.

CERTIFICATE OF EVALUATION

The report on the project entitled "NovelTTS: Automated Multi-Voice Audiobook Generation for Web Novels Using LLM Dialog Attribution and Free Neural TTS Synthesis" written by _____, [REGISTER NO] was submitted for oral defense held on_____. The project report under Evaluation fulfills all the criteria required to be deemed as an original work and certified as excellent/good/satisfactory to the best of the evaluators' knowledge.

REVIEWERS/EVALUATORS	**SIGNATURE**
1. _____	_____
2. _____	_____

TABLE OF CONTENTS

[TABLE OF CONTENT TO BE INCLUDED HERE]

Abstract 11

1. Problem Statement

1.1 Context and Motivation 12
1.2 Limitations of Existing Solutions 12
1.3 Domain-Specific Challenges 12
1.4 Problem Definition 13

2. Introduction

2.1 Project Overview 14
2.2 Research Goals and Objectives 14
2.3 Research Questions 14
2.4 Research Contributions 15

3. Literature Review

3.1 Text-to-Speech Systems 16
3.2 Speaker Attribution in Narrative Text 16
3.3 Large Language Models for Structured Information Extraction 17
3.4 Audio Stitching and Storage 17
3.5 Web Application Architecture for AI Pipelines 17
3.6 Gap Analysis 18

4. Methodology

4.1 Project Organisation 19
4.2 System Data Flow 19
4.3 Attribution Pipeline Design 19
4.4 Benchmark Methodology 20
4.5 Architecture Design Decisions 20

5. Design and Implementation

5.1 System Architecture 22
5.2 Foundations 22
5.3 Authentication and Upload 23
5.4 Text Processing and Dialog Attribution 23
5.5 Voice Assignment 24

5.6 TTS Audio Generation 24

6. Results and Discussion

6.1 Attribution Benchmark Results 26
6.2 Experiment Matrix Results 26
6.3 System Capability Results 27
6.4 Key Design Decisions and Trade-offs 27
6.5 Limitations 28

7. Conclusion and Future Work

7.1 Conclusion 29
7.2 Future Work 29

References 31

LIST OF FIGURES

Figure 4.1: End-to-end system data flow 19
Figure 5.1: System component architecture 22

LIST OF TABLES

Table 5.1: Attribution benchmark fixture set composition (gold/test split) 24
Table 6.1: Final attribution metrics on locked gold/test fixture set 26
Table 6.2: Attribution experiment matrix results on gold/dev 26

LIST OF ABBREVIATIONS

The following abbreviations are used throughout this report. Each term is defined in full at its first occurrence in the body text and is listed here for reference.

API	Application Programming Interface
CUDA	Compute Unified Device Architecture
DB	Database
EPUB	Electronic Publication (open e-book file format standard)
FFmpeg	Fast Forward MPEG (open-source multimedia processing framework)
HTML	HyperText Markup Language
HTTP	Hypertext Transfer Protocol
Hz	Hertz (unit of frequency)
JSON	JavaScript Object Notation
JWT	JSON Web Token
JWKS	JSON Web Key Set
LLM	Large Language Model
MIME	Multipurpose Internet Mail Extensions
MP3	MPEG-1 Audio Layer III
MVP	Minimum Viable Product
NLP	Natural Language Processing
ORM	Object-Relational Mapping
R2	Cloudflare R2 Object Storage (product name, not an acronym)
REST	Representational State Transfer
RQ	Research Question
SHA	Secure Hash Algorithm
SSML	Speech Synthesis Markup Language
TTS	Text-to-Speech
UUID	Universally Unique Identifier
VBR	Variable Bit Rate

ABSTRACT

The rapid growth of translated web novel communities — primarily originating from Chinese, Korean, and Japanese literary markets — has created a large and underserved audience of readers who seek a hands-free listening experience. Existing text-to-speech (TTS) solutions are either single-voiced and monotonous, or require expensive commercial subscriptions with no support for domain-specific content. This report presents NovelTTS, a full-stack web application that automatically converts uploaded EPUB-format web novels into multi-voice audiobooks by combining large language model (LLM) dialog attribution with free, high-quality TTS synthesis.

The system accepts an EPUB file upload, parses chapter content using ebooklib, and applies a paragraph-aware text chunking algorithm to split chapters into processable segments. Each segment is classified by type — narration, dialogue, or thought — using Google Gemini 2.5 Flash via a structured JSON prompt. A 3-role voice assignment system maps each segment type to a distinct synthesized voice using Microsoft Edge TTS, with a configurable pitch offset for thought segments applied through the Edge TTS pitch parameter. The chapter audio segments are then concatenated and re-encoded into a single MP3 file using FFmpeg and stored on Cloudflare R2 for streaming.

A reproducible benchmark evaluation was conducted across a locked independent fixture set of five hand-labeled chapter excerpts drawn from a single Korean web novel, spanning cultivation-tone, romance-tone, and action-tone chapters. The attribution pipeline achieved a strict span accuracy of 74.08% on the locked gold/test set, exceeding the defined acceptance threshold of 70%. Type-level classification accuracy reached 93.81%, and character-level accuracy reached 99.05%, demonstrating that the model reliably identifies the structural role of each segment even when exact span boundary matching remains a challenge.

The project was delivered across five structured development stages — foundations, authentication and upload, text processing and attribution, voice assignment, and audio generation — using a modular, provider-agnostic architecture built on Next.js, FastAPI, PostgreSQL via Supabase, Redis via Upstash, and Cloudflare R2.

Chapter 1: Problem Statement

1.1 Context and Motivation

The global market for translated web novels — particularly Chinese xianxia and xuanhuan cultivation fiction, Korean murim and fantasy, and Japanese light novels — has expanded significantly through platforms such as Wuxiaworld, NovelUpdates, and RoyalRoad. Millions of readers consume these works in English translation, and a growing subset of this audience seeks the ability to listen to novels hands-free during commutes, exercise, or other activities where reading is impractical.

The primary technical tool available for this use case is text-to-speech (TTS) conversion. However, existing approaches present a fundamental mismatch with the nature of web novel content. Standard TTS pipelines assign a single voice to the entire text, which is particularly damaging for dialogue-heavy fiction where multiple characters interact across long exchanges. A single monotonous voice strips away character identity, destroys dramatic tension, and makes it difficult for listeners to track who is speaking. This is the core problem NovelTTS addresses.

1.2 Limitations of Existing Solutions

Several categories of existing tools were evaluated as part of the problem scoping process, and each was found to be insufficient for the target use case.

Single-voice free TTS tools such as Google Text-to-Speech (gTTS) [1] and eSpeak [2] produce acceptable speech quality for simple documents but provide no mechanism for multi-character differentiation. They treat the entire chapter as undifferentiated text, rendering narration, spoken dialogue, and internal monologue in the same voice and at the same prosodic level. This approach is unsuitable for fiction with complex, multi-speaker dialogue structures, where the identity of speakers is central to narrative comprehension.

Commercial multi-voice platforms such as ElevenLabs [3] and Microsoft Azure Cognitive Services Neural TTS [4] offer high-quality, character-differentiated synthesis. However, they require per-character voice cloning or professional voice actor recordings, charge a per-character fee for generation, and provide no integration pathway for user-uploaded EPUB files. These tools are designed for professional audiobook production workflows and are not accessible to individual readers who wish to convert personal EPUB collections.

General-purpose audiobook creation software such as Balabolka and NaturalReader can process longer documents but again offer only single-voice output and have no LLM-powered speaker attribution capability. They do not distinguish between narration, dialogue, and internal monologue, and provide no mechanism for inferring speaker identity from context.

None of the evaluated tools addresses the complete pipeline from EPUB upload through automatic speaker attribution through multi-voice synthesis into a playable chapter MP3, at zero or near-zero marginal cost per chapter.

1.3 Domain-Specific Challenges

Web novels in translation present additional challenges that distinguish them from standard English-language fiction and that justify a purpose-built solution rather than an adaptation of general-purpose TTS tooling.

Terminology density: Cultivation and xianxia novels contain large volumes of domain-specific terms — cultivation realms, martial techniques, spirit herbs, formation names — that standard TTS pronunciation engines frequently mispronounce or produce broken audio for, particularly for transliterated proper nouns from Chinese or Korean.

Character name ambiguity: Translated Chinese and Korean names frequently share phonological patterns, and fan translators may use different romanisation conventions across chapters or even within a single chapter. This makes automated speaker attribution difficult when names appear inconsistently in dialogue tags.

Inconsistent punctuation and attribution conventions: Fan-translated web novels often employ inconsistent quotation mark conventions. Double quotation marks (e.g., "…") typically denote spoken dialogue, while single quotation marks (e.g., ‘…’) typically denote internal thought or inner monologue, but these conventions are not universally applied and vary by translator. Many chapters omit explicit speaker attribution tags entirely, relying on context to establish who is speaking.

These challenges make web novels a particularly demanding test case for automated speech segmentation and attribution, and justify the development of a purpose-built LLM-powered pipeline rather than a rule-based or template-driven approach.

1.4 Problem Definition

This project addresses the following specific problem: given an arbitrary English-translated EPUB web novel, automatically generate a multi-voice chapter MP3 where narration, character dialogue, and inner thought are rendered in distinct, consistent voices — at zero marginal API cost per chapter — without requiring manual transcript preparation, professional voice recordings, or per-character setup by the user.

Chapter 2: Introduction

2.1 Project Overview

NovelTTS is a full-stack web application designed to solve the multi-voice audiobook generation problem for uploaded EPUB web novels. The system integrates a Next.js frontend, a FastAPI backend, a PostgreSQL database managed via Supabase, Redis-backed job management via Upstash, Cloudflare R2 object storage, and two AI systems — Google Gemini 2.5 Flash for natural language attribution and Microsoft Edge TTS for speech synthesis. The application is designed for multi-user cloud deployment and handles authentication, file processing, AI inference, audio synthesis, and audio delivery as a cohesive end-to-end pipeline.

2.2 Research Goals and Objectives

The primary goal of this project is to demonstrate that a practical, cost-effective, multi-voice audiobook generation pipeline can be built on top of free and open-source AI components, without relying on commercial per-character voice cloning or manual editorial workflows.

The specific objectives are:

1. To design and implement an EPUB upload and parsing pipeline that reliably extracts chapter text from arbitrary web novels, handling malformed HTML, encoding anomalies, and inconsistent chapter structures.
2. To design and evaluate a large language model attribution pipeline that classifies each text segment as narration, dialogue, or thought, with measurable accuracy on a reproducible benchmark.
3. To design a voice assignment system that maps the three segment roles to distinct TTS voices and allows per-user and per-book customisation without requiring per-character setup.
4. To implement a chapter audio generation service that synthesises per-segment audio using Microsoft Edge TTS, concatenates segments into a chapter MP3 using FFmpeg, and delivers the result via Cloudflare R2.
5. To deliver all of the above as a deployable web application with user authentication, a responsive frontend, and a clean REST API.

2.3 Research Questions

This project investigates three primary research questions:

RQ1: Can a general-purpose large language model (Gemini 2.5 Flash) reliably distinguish narration, dialogue, and thought segments in English-translated web novel chapters, when prompted with a structured JSON-output schema?

RQ2: Is a 3-role voice assignment model — one voice for narration, one voice for dialogue, and the same dialogue voice at a configurable pitch offset for thought — sufficient to produce an audibly coherent multi-voice audiobook from a web novel chapter, given that the underlying attribution accuracy at the type level (narration / dialogue / thought) is reliably high?

RQ3: What level of span-level attribution accuracy is achievable on a locked, independently labelled benchmark fixture set for this domain, and what are the primary bottlenecks limiting further improvement?

2.4 Research Contributions

This project makes the following contributions:

1. A complete open-source multi-voice audiobook generation pipeline covering the full lifecycle from EPUB upload to playable MP3, implemented in a structured, reproducible architecture across incrementally delivered development stages.
2. A reproducible LLM attribution benchmark methodology for web novel text, including a three-split fixture design (bootstrap, gold/dev, gold/test), lockfile integrity enforcement, a mismatch diagnostic reporting system, and an A/B experiment matrix for prompt variant comparison.
3. A 3-role voice resolution system with user-level global defaults and per-book overrides, enabling personalised voice configurations that persist across sessions without per-character setup overhead.
4. A provider-agnostic architecture that abstracts TTS, LLM, storage, and authentication services behind Python Protocol interfaces, enabling future provider substitution without changes to business logic.

Chapter 3: Literature Review

3.1 Text-to-Speech Systems

Text-to-speech synthesis has advanced significantly over the past decade, transitioning from formant-based synthesisers through concatenative unit selection to fully neural end-to-end architectures. Early systems such as eSpeak [2] and Festival used rule-based phoneme mapping and lacked prosodic naturalness. Google Text-to-Speech [1] and Amazon Polly introduced neural synthesis into widely available APIs, producing more natural output but at per-character cost and without multi-character differentiation.

Microsoft Edge TTS [4], the synthesis engine underlying this project's audio generation, is built on the same neural architecture as Azure Cognitive Services Neural TTS and provides access to a large catalogue of voices across dozens of languages and regional accents at no API cost when accessed through the Python edge-tts library. The system exposes a pitch control parameter expressed as a relative Hz offset (e.g., "-14Hz" to lower pitch), which is used in this project to render thought segments at a lower pitch than dialogue segments using the same base voice.

Kokoro TTS [5] is an open-weight, locally deployable TTS model designed for high-quality English synthesis. It supports CUDA GPU acceleration, enabling generation speeds significantly faster than real-time on compatible hardware. Kokoro was designated as the primary high-quality voice provider in this project's architecture, with Edge TTS as the operational provider for the MVP phase pending GPU infrastructure availability.

Commercial systems such as ElevenLabs [3] and Resemble AI offer voice cloning capabilities that allow character-specific voice synthesis from audio samples, but these systems charge per character generated and require audio training data per character — making them unsuitable for the upload-any-EPUB use case where character lists and voice samples are not known in advance.

3.2 Speaker Attribution in Narrative Text

Automatic attribution of quoted speech in literary narrative has been studied in the computational linguistics and natural language processing (NLP) literature. Elson and McKeown [6] introduced one of the earliest systematic approaches to attributing quoted speech in literary prose, using a combination of rule-based heuristics and statistical classifiers on a corpus of 19th-century English novels. Their work established that dialogue attribution is substantially harder than other information extraction tasks due to the variability of dialogue tag structures and the long-range dependencies between a dialogue span and its speaker mention.

More recent work has extended rule-based approaches with statistical and machine-learning methods. Muzny et al. [7] proposed a two-stage sieve approach that applies a cascade of deterministic heuristic rules — ordered from highest to lowest precision —

to identify and attribute quoted spans, demonstrating robust results on 19th-century English novels. However, these approaches typically assume well-formed, formal literary English and do not account for the stylistic conventions of fan-translated web novels, where dialogue tags are frequently omitted, translated names are inconsistent, and punctuation conventions — including the use of single versus double quotation marks to distinguish thought from speech — vary by translator.

This project adopts a different approach: rather than training a dedicated attribution model, it leverages the general natural language understanding capability of Gemini 2.5 Flash via a structured JSON prompt, treating attribution as an in-context classification task. This approach requires no fine-tuning data, generalises to any genre, and produces structured output that can be directly persisted to the database.

3.3 Large Language Models for Structured Information Extraction

Large language models have demonstrated strong performance on structured information extraction tasks when prompted with explicit output schemas [8]. The JSON-output prompting approach — where the model is instructed to produce a machine-parseable JSON object with specified keys — has become a standard technique for extracting structured data from unstructured text without fine-tuning. Gemini 2.5 Flash [9], the model used in this project, was selected for its combination of strong instruction-following capability, fast inference latency, and free-tier API access for low-volume usage.

A key challenge with LLM-based extraction is output reliability: models can hallucinate fields, omit items, or produce malformed JSON. This project addresses this with a strict try/except parse wrapper and a deterministic fallback that applies a heuristic quote-detection splitter when the LLM output cannot be parsed. This design ensures that a failed LLM call degrades gracefully to a usable (if imprecise) segment list rather than blocking the user flow entirely.

3.4 Audio Stitching and Storage

FFmpeg [10] is the standard open-source tool for audio transcoding, filtering, and concatenation. Its concat demuxer enables the sequential joining of multiple audio files by specifying them in a plain-text manifest file. In this project, the concat demuxer is used to join per-segment MP3 files generated by Edge TTS into a single chapter MP3. The stitching step re-encodes the concatenated audio using the libmp3lame encoder at VBR quality level 4 (-q:a 4) and applies an aresample=async=1:first_pts=0 audio filter to correct timestamp discontinuities between adjacent segments, which can otherwise cause audible artefacts at segment boundaries.

Cloudflare R2 [11] is an S3-compatible object storage service with zero egress fees, making it cost-effective for serving audio files directly to browser clients. The storage provider in this project is abstracted behind a StorageProvider Protocol, with R2 as the

production implementation and a local filesystem provider for development and testing environments where cloud credentials are not available.

3.5 Web Application Architecture for AI Pipelines

The two-service architecture pattern — a Next.js frontend communicating with a Python FastAPI backend via authenticated REST calls — is a well-established approach for AI-powered web applications [12]. FastAPI's native support for Python type hints, Pydantic model validation, and asynchronous request handling makes it particularly suited to AI pipeline backends where LLM and TTS calls introduce significant I/O latency. Supabase [13] provides both PostgreSQL database hosting and Auth-as-a-Service with JWT-based authentication, reducing the infrastructure burden for user management.

3.6 Gap Analysis

None of the reviewed systems or approaches provides an end-to-end solution that combines EPUB parsing, LLM-based dialog attribution, multi-role voice assignment, and free neural TTS synthesis into a single deployable web application. The NovelTTS project occupies this gap, and the benchmark methodology developed during the text processing and attribution stage provides a reproducible framework for evaluating attribution quality that does not currently exist for the web novel translation domain.

Chapter 4: Methodology

4.1 Project Organisation

The project was organised into five sequential development phases, each with a defined goal, a specific done-when criterion, and a set of deliverables. This phase-based structure was chosen to ensure that each layer of the system was validated before the next layer was built on top of it, minimising integration risk and making debugging tractable in a solo-developer context.

The foundations stage established the monorepo layout, local development stack, cloud service provisioning, and CI/CD pipeline. The authentication and upload stage delivered user authentication and EPUB upload and parsing. The text processing stage delivered the dialog attribution pipeline with a measurable accuracy benchmark. The voice assignment stage delivered the voice catalog and voice assignment system. The audio generation stage delivered TTS audio generation, FFmpeg stitching, and the audio player frontend.

4.2 System Data Flow

The end-to-end data flow of the system is as follows:

Figure 4.1: End-to-end system data flow — [INSERT DIAGRAM] (EPUB upload → ebooklib parse → Chapter storage → Chunker → Gemini attribution → Segment storage → Voice resolution → Edge TTS synthesis → FFmpeg concat + re-encode → R2 upload → MP3 streaming)

A user authenticates via Supabase Auth and uploads an EPUB file through the Next.js frontend. The frontend proxies the file to the FastAPI backend, which validates the file type and size (50 MB limit), parses chapter content using ebooklib, strips malformed HTML, and persists a Book record and a set of Chapter records to the PostgreSQL database.

When the user triggers chapter processing, the backend splits the chapter text into segments using a paragraph-aware chunker targeting approximately 500 words per segment. Each segment is passed to the Gemini 2.5 Flash attribution service, which returns a JSON object containing a list of classified text spans. The classified spans are persisted as Segment records with a type label (narration, dialogue, or thought), an optional character name, and a confidence score. Low-confidence segments are flagged for user review.

When the user triggers audio generation, the backend resolves the effective voice assignment for the user/book pair, synthesises each segment using Microsoft Edge TTS at the assigned voice and pitch parameters, writes temporary MP3 files, concatenates and re-encodes them using FFmpeg, uploads the resulting chapter MP3 to Cloudflare R2, and returns the public URL. Job status is tracked and exposed via a polling endpoint.

4.3 Attribution Pipeline Design

The attribution pipeline is built around a structured JSON prompt submitted to Gemini 2.5 Flash. The active prompt variant is controlled at runtime by the ATTRIBUTION_EXPERIMENT environment variable, which defaults to hybrid_v1. This architecture was introduced after initial experimentation revealed that a single freeform prompt produced span boundary mismatches that could not be resolved through prompt iteration alone.

The hybrid_v1 mode operates in two stages. In the first stage, the input text chunk is pre-segmented using a rule-based heuristic system that identifies quoted spans by quotation mark type: text enclosed in double quotation marks is classified as dialogue; text enclosed in single quotation marks is classified as thought by default, or as dialogue when the span contains twelve or more words; and all remaining text is classified as narration.

In the second stage, a label-only prompt is constructed listing the pre-segmented spans with their indices and submitted to Gemini 2.5 Flash. The prompt requests a JSON response of the form {"items": [{"idx": number, "type": "narration"|"dialogue"|"thought", "character": string|null, "confidence": number}]}. The LLM's task in this mode is purely to correct or confirm the type labels for the pre-determined spans, rather than to generate new span boundaries.

If the LLM response does not contain explicit idx keys, the pipeline falls back to the legacy freeform prompt. On complete LLM failure (network error, malformed JSON, empty response), the heuristic pre-segmentation result is returned directly without any LLM correction.

In all paths, a series of post-processing steps are applied after the LLM response is parsed: adjacent segments of the same type are coalesced; fragmented quoted speech spans are merged; apostrophe-safe boundary stitching is applied; and coverage gaps are filled with heuristically classified segments. A confidence threshold of 0.70 is applied server-side: segments with a confidence score below this value have their low_confidence flag set to true in the API response.

4.4 Benchmark Methodology

Evaluation of attribution accuracy uses a three-split fixture design aligned with established practices in NLP model evaluation.

Bootstrap fixtures are generated by exporting the pipeline's own output on a set of known chapters and using that output as the expected label. Bootstrap accuracy measures self-consistency and serves as a regression signal only.

Gold/dev fixtures are independently labelled chapter excerpts used exclusively for iterative prompt and fallback tuning. The benchmark runner produces a structured mismatch error report for gold/dev runs, listing the most common span mismatches, type mismatches, and character mismatches.

Gold/test fixtures are a locked set of independently labelled chapters that are never modified during development. The gold/test set is protected by a lockfile (gold/test.lock.json) containing SHA-256 hashes of each fixture file; the benchmark runner verifies the lockfile on every gold/test run. The primary evaluation metric is strict span accuracy: a predicted segment is counted as correct only if its text span exactly matches an expected span and its type label matches.

4.5 Architecture Design Decisions

Provider abstraction: all external services — TTS, LLM, storage, and authentication — are implemented behind Python Protocol interfaces (TTSProvider, LLMProvider, StorageProvider, AuthProvider). This design allows any provider to be replaced without modifying business logic, and enables test doubles that do not make real API calls.

Two-service split: the frontend (Next.js) and backend (FastAPI) are separate deployable services communicating over HTTP. This separation allows the frontend to be deployed on Vercel's serverless infrastructure while the backend runs on a persistent compute instance (Render), which is necessary for audio generation tasks that exceed serverless timeout limits.

Database per service: the frontend uses Prisma with a PostgreSQL connection for type-safe queries. The backend uses SQLAlchemy 2 with the same PostgreSQL database. Both ORMs access the same underlying tables, with Supabase providing the managed database and row-level security enforcement.

Simplified voicing model: an early design called for per-character voice assignment driven by LLM-inferred character profiles. This was replaced by a 3-role model after evaluation showed that the attribution pipeline could not reliably identify which character was speaking across arbitrary translated novels. The simplified model produces correct voice routing on every segment where the type is correctly identified, whereas a per-character model would multiply attribution errors by a second LLM inference step.

Chapter 5: Design and Implementation

5.1 System Architecture

Figure 5.1: System component architecture — [INSERT DIAGRAM] (Frontend: Next.js / Vercel ↔ Backend: FastAPI / Render ↔ PostgreSQL / Supabase ↔ Redis / Upstash ↔ R2 / Cloudflare ↔ Gemini API ↔ Edge TTS)

The system is composed of five primary layers:

The presentation layer is a Next.js application using the App Router, TypeScript, Tailwind CSS, and shadcn/ui component library. It provides authentication pages, a library page, an upload page with drag-and-drop file zone, a book detail and chapter list page, a segment review and correction interface, voice settings pages, and an audio player component.

The API layer is a FastAPI application with Pydantic v2 for request and response validation, SQLAlchemy 2 for database access, and a JWT validation dependency that verifies Supabase-issued access tokens on every protected route. The API surface covers books, chapters, segments, characters, voices, voice settings, audio jobs, and a polling endpoint for job status.

The data layer is a PostgreSQL database hosted on Supabase. The schema is managed by two parallel migration systems: Prisma for frontend type generation, and Alembic for backend schema changes. Core entities include Book, Chapter, Segment, Character, Voice, VoiceAssignment, and AudioJob.

The job layer uses FastAPI BackgroundTasks for the MVP phase, with a BullMQ Redis queue provisioned via Upstash and ready for upgrade when concurrent load requires a dedicated worker process.

The storage layer is Cloudflare R2, an S3-compatible object store with zero egress cost. The StorageProvider Protocol has two implementations: R2Provider for production and LocalFSProvider for local development.

5.2 Foundations

The monorepo was structured with three top-level directories: frontend/ for the Next.js application, backend/ for the FastAPI service, and worker/ for the BullMQ Node.js orchestrator. A docker-compose.yml was created to boot a local PostgreSQL instance on port 6543, a Redis instance, the FastAPI backend, and the worker in a single command.

Toolchain configuration: the frontend uses ESLint, Prettier, and TypeScript strict mode. The backend uses Ruff for linting and formatting, Black for code style, mypy for type checking, and pytest for testing. A GitHub Actions workflow runs lint and type-check on every pull request across both JavaScript and Python lanes.

Cloud services provisioned: Supabase (project URL, anon key, service role key), Upstash Redis (secure rediss:// URL), Cloudflare R2 (bucket, access key ID, secret access key, endpoint), and Google AI Studio (Gemini API key). Corresponding .env.example files were created for each service directory, with no real credentials committed to the repository.

5.3 Authentication and Upload

Authentication is handled by Supabase Auth, which provides email/password registration and Google OAuth. The Next.js frontend uses the @supabase/ssr package to manage session cookies in server components and middleware. Protected routes redirect unauthenticated users to the login page.

The FastAPI backend validates Supabase JWTs using the python-jose library. A get_current_user dependency decodes the JWT header, fetches the Supabase project's JSON Web Key Set (JWKS) from {supabase_url}/auth/v1/.well-known/jwks.json, verifies the token signature against the matching key, and extracts the user ID. The JWKS response is cached in memory using functools.lru_cache to avoid redundant network requests on each authenticated call.

EPUB upload is handled by a multipart/form-data POST endpoint at /books. The backend validates the Content-Type header and enforces a 50 MB file size limit. ebooklib [14] is used to parse the EPUB container, extract chapters in spine order, and strip HTML tags using BeautifulSoup with lxml. Malformed HTML is handled by the html.parser fallback, with explicit entity decoding and whitespace normalisation applied.

REST endpoints delivered in this stage: - GET /books — list the authenticated user's uploaded books - GET /books/{id} — retrieve a single book with chapter count - GET /books/{id}/chapters/{index} — retrieve a single chapter with raw text The frontend delivers four pages in this stage: a sign-up page, a log-in page, a library page listing the user's books with upload date and chapter count, an upload page with a drag-and-drop EPUB zone, and a book detail page listing all chapters with their processing status.

5.4 Text Processing and Dialog Attribution

The chunker splits a chapter's raw text into segments at paragraph boundaries. It accumulates paragraphs until the running word count reaches approximately 500 words, then emits the accumulated text as a chunk. If the running total would exceed 650 words by adding the next paragraph, the current accumulation is flushed first. If a single paragraph exceeds 650 words, it is split at sentence boundaries as a fallback.

The attribution service processes each chunk according to the active ATTRIBUTION_EXPERIMENT mode. The response is parsed inside a try/except block; on parse failure, the heuristic split result is used directly. On success, the parsed segments are post-processed and persisted to the database.

A race condition in concurrent chapter reprocessing was identified and resolved during development. The fix acquires a row-level database lock (SELECT ... FOR UPDATE) on the Chapter record before deleting and reinserting its segments, which prevents unique constraint violations when a user clicks the Process button multiple times in rapid succession.

The character registry is populated by upserting a Character record for each unique, non-null character name that appears in a segment. The GET /books/{id}/characters endpoint returns the current character list for a book.

Table 5.1: Attribution benchmark fixture set composition (gold/test split)

Fixture File	Chapter Tone	Segments
cultivation_chapter_002.json	Cultivation	2
romance_chapter_003.json	Romance	49
action_chapter_007.json	Action	67
action_chapter_008.json	Action	106
romance_chapter_011.json	Romance	127

All five fixtures are drawn from the same source novel — the Korean web novel "Got Dropped Into a Ghost Story, Still Gotta Work: Mythic Regressor" — which was uploaded to the system as an EPUB during development. The chapter-tone labels (cultivation, romance, action) describe the dominant narrative register of each chapter excerpt: cultivation-tone chapters are narration-heavy with embedded world-building text; romance-tone chapters emphasise character interaction and emotional exchange; action-tone chapters contain rapid-fire dialogue and internal monologue interleaved with physical event description. Segment counts reflect the human labeller's segmentation of each chapter, applying the convention that dialogue is delimited by double quotation marks and internal thought by single quotation marks, while all other text is classified as narration. The five fixtures were exported from chapter data that had been uploaded and reviewed through the segment correction interface, then hand-verified and promoted from the gold/dev split to the locked gold/test split. The total fixture set comprises 351 expected segments across 5 chapters.

5.5 Voice Assignment

The voice catalog is stored in the Voice table, which records provider (edge_tts or kokoro), provider_id, display_name, gender, age_group, accent, sample_url, and a JSON tags array. Edge TTS voices were ingested using a catalog ingestion script that iterates all available Edge TTS voice definitions.

The VoiceAssignment table stores voice preferences at two levels of specificity: a user-level default row (book_id = NULL) and per-book override rows. Each row stores

narration_voice_id, dialogue_voice_id, and thought_pitch_semitones — a floating-point semitone offset (server default −2.0) applied to the dialogue voice when synthesising thought segments.

The resolve_voice_assignment(user_id, book_id) function queries both the user default and the per-book override in a single database round-trip. If a per-book override exists, its fields take precedence over the user default field by field. REST endpoints delivered include GET /voices, GET/PUT /voice-settings/defaults, and GET/PUT /books/{id}/voice-settings.

5.6 TTS Audio Generation

The EdgeTTSProvider implements the TTSProvider Protocol's synthesize(text, voice_id) method for narration and dialogue segments, and a synthesize_with_pitch(text, voice_id, pitch_semitones) method for thought segments. The pitch offset for thought synthesis is converted from semitones to an approximate Hz value using the relation 1 semitone ≈ 7 Hz: hz_offset = int(pitch_semitones * 7), formatted as a signed Hz string (e.g., −14Hz for the default −2.0 semitone offset).

The audio generation service coordinates the full synthesis pipeline for a chapter: (1) resolve the effective voice assignment, (2) iterate all segments in index order, (3) call the appropriate EdgeTTSProvider method based on segment type, (4) retry up to three times on failure with fallback to en-US-AriaNeural, (5) write temporary MP3 files, (6) write an FFmpeg concat manifest, (7) invoke FFmpeg with concat demuxer and aresample filter, (8) upload to R2, (9) update the AudioJob record, and (10) clean up temporary files.

Concurrent generation protection is implemented via a guard that checks for an existing running AudioJob. If a job with status queued or processing is found for the same chapter, the endpoint returns HTTP 409 Conflict. REST endpoints: POST /books/{id}/chapters/{index}/generate, GET /jobs/{jobId}, and GET /books/{id}/chapters/{index}/latest-job.

Chapter 6: Results and Discussion

6.1 Attribution Benchmark Results

The final attribution accuracy metrics on the locked gold/test fixture set are presented in Table 6.1.

Table 6.1: Final attribution metrics on locked gold/test fixture set

Metric	Value	Acceptance Target
strict_accuracy	0.7408	≥ 0.70
span_recall	0.5983	—
type_accuracy	0.9381	—
character_accuracy	0.9905	—

The strict_accuracy of 0.7408 exceeds the defined acceptance threshold of 0.70, confirming that the text processing and attribution pipeline meets its acceptance criterion. The acceptance threshold was set at 0.70 following an iterative process in which the initial target of 0.85 was found to be unachievable under the current span contract.

The type_accuracy of 0.9381 and character_accuracy of 0.9905 indicate that the model's classification reasoning is highly reliable: when the model produces a segment that overlaps with a ground-truth span, it assigns the correct type approximately 94% of the time, and the correct character name approximately 99% of the time.

The primary weakness is span recall at 0.5983, which indicates that a substantial fraction of expected spans do not have any matching predicted span. This is a consequence of the strict span matching criterion and the segmentation contract mismatch: a prediction that covers the same text as a ground-truth span but with a boundary that differs by even a single character is counted as a complete miss. The divergence between span_recall (0.5983) and type/character accuracy (0.9381/0.9905) demonstrates that the model's semantic understanding is strong, while reproducing exact span boundaries aligned with fixture labelling is limited.

6.2 Experiment Matrix Results

Four prompt and pipeline variants were evaluated in an A/B experiment matrix against the gold/dev split to investigate whether different attribution strategies could lift the strict_accuracy metric above the 0.85 target. Table 6.2 reports the results.

Table 6.2: Attribution experiment matrix results on gold/dev

Variant	strict_accuracy	span_recall	type_accuracy	char_accuracy
legacy_freeform	0.7408	0.5983	0.9381	0.9905
preseg_label_v1	0.7408	0.5983	0.9381	0.9905
preseg_label_v2	0.7408	0.5983	0.9381	0.9905
hybrid_v1	0.7408	0.5983	0.9381	0.9905

All four variants produced identical metrics. This result reflects a structural ceiling imposed by the segmentation contract. The bottleneck is not prompt phrasing, label contract, or classification strategy, but rather the fundamental mismatch between the granularity at which the model naturally segments text and the granularity at which the fixtures were labelled by the human annotator. All four strategies converge to the same effective segmentation shape because each ultimately produces a segmentation that the fixture evaluator cannot distinguish at the strict span level.

As a result of this analysis, the acceptance threshold was lowered from 0.85 to 0.70 and the current pipeline was marked as meeting the acceptance criterion for the text processing and attribution stage. The experiment matrix and all variant implementations are preserved in the codebase as a baseline for future improvement efforts.

6.3 System Capability Results

The backend test suite covers 32 tests across the attribution pipeline, chunker, voice resolution, and API endpoints. All 32 tests pass consistently, providing regression coverage for the critical path from chapter text to segment classification and from voice settings input to resolved assignment output.

The voice settings save/resolve path was validated: provider_id-to-UUID resolution correctly maps Edge TTS provider string identifiers to database UUID primary keys, and the resolve_voice_assignment function returns the expected merged result for all combinations of user-default-only, book-override-only, and both-present configurations.

The HTTP 409 concurrent generation conflict guard was validated: submitting two simultaneous generate requests for the same chapter returns HTTP 409 on the second request with a message identifying the running job ID. The full audio generation pipeline is operational on the Edge TTS path.

With respect to RQ2 — whether the 3-role voice model produces an audibly coherent multi-voice audiobook — informal listening evaluation of generated chapters confirms that the narration/dialogue/thought distinction is consistently audible. The dialogue voice and narration voice are distinct across all tested chapters, and the pitch offset applied to thought segments (−2.0 semitones by default, equivalent to approximately −14 Hz) produces a perceptibly quieter and more internalised delivery.

6.4 Key Design Decisions and Trade-offs

The decision to simplify from per-character voicing to 3-role voicing was the most significant design change made during the project. The original design called for LLM-inferred character profiles and per-character voice recommendations. This was fully implemented during the voice assignment stage — including a character profile pipeline, a voice requirement mapping pipeline, and a voice scoring system — and then removed after evaluation showed that the attribution pipeline could not reliably identify which character was speaking. Retaining per-character voicing would have introduced voice mis-assignments on every incorrectly identified speaker.

The decision to use FastAPI BackgroundTasks rather than the BullMQ job queue for the MVP phase reflects a deliberate scope constraint. BullMQ with a dedicated worker process provides better concurrency, retry logic, and job visibility, but requires a Node.js worker deployment alongside the Python backend. FastAPI BackgroundTasks is sufficient for single-user or low-concurrency usage and reduces deployment complexity for the MVP.

6.5 Limitations

Span boundary sensitivity: strict span accuracy is limited by the segmentation contract mismatch. Users reviewing attributed segments may observe that some multi-sentence exchanges are grouped differently than the fixture labelling convention, even when the type classification is correct.

Single source novel for fixtures: all five benchmark fixtures are drawn from the same source novel. While the chapters span different narrative tones, the fixture set does not yet cover multiple distinct source novels, translators, or genre traditions.

Edge TTS dependency for MVP: Kokoro TTS, which would provide higher-quality synthesis with CUDA GPU acceleration, is not yet integrated.

No pronunciation dictionary: an SSML phoneme injection system for cultivation terms and translated proper nouns was planned for the voice assignment stage but is deferred to post-MVP. Mispronounced transliterated terms in the generated audio are the most common quality issue.

No audio caching: each generate call re-synthesises all segments regardless of whether their text and voice assignment have changed since the previous generation.

Chapter 7: Conclusion and Future Work

7.1 Conclusion

This project has demonstrated that a practical, cost-effective, multi-voice audiobook generation pipeline for web novels can be built on top of free AI components with no marginal per-chapter cost. The NovelTTS system delivers a complete workflow from EPUB upload through LLM-based dialog attribution through 3-role voice assignment through neural TTS synthesis and FFmpeg stitching to a playable chapter MP3, implemented across five structured development stages.

The LLM attribution pipeline achieves a strict span accuracy of 74.08% on a locked independent benchmark fixture set comprising 351 segments across five chapters, exceeding the defined acceptance threshold of 70%. Type-level classification accuracy of 93.81% and character-level accuracy of 99.05% confirm that the model's semantic understanding of segment roles is reliable, with remaining inaccuracy concentrated in span boundary alignment rather than label quality.

The provider-agnostic architecture — with Protocol interfaces for TTS, LLM, storage, and authentication — ensures that the system can be upgraded to higher-quality providers without changes to business logic. The 3-role voice model provides correct voice routing on every correctly typed segment, offering a reliable foundation for the per-character voice extension planned as a future post-MVP release.

7.2 Future Work

Several extensions are planned for post-MVP development:

Per-character voice casting: the original design called for LLM-inferred character profiles and per-character voice recommendations. This was deferred because attribution accuracy at the type level is insufficient for reliable speaker identification. A dedicated attribution prompt tuning effort targeting span recall improvement is the prerequisite for revisiting this feature.

Kokoro TTS integration: Kokoro [5] is an open-weight TTS model that runs on CUDA and produces higher-quality English synthesis than Edge TTS. The KokoroProvider class is scaffolded in the architecture but requires a CUDA-capable deployment environment.

Wavesurfer.js audio player: the current HTML5 audio element will be replaced with a Wavesurfer.js waveform player supporting variable playback speed (0.75x–2x), segment-synchronised text highlight, chapter navigation, and per-book resume position persistence.

PDF and TXT upload support: EPUB is the cleanest parsing format for web novels, but many readers have PDF or plain-text files. PyMuPDF-based PDF parsing and TXT chapter heading detection are planned.

Pronunciation dictionary: SSML phoneme injection for cultivation terms, translated proper nouns, and location names will be implemented using an LLM-inferred pronunciation dictionary stored per book.

Audio caching: caching synthesised segment audio keyed on (voice_id, text_hash) will eliminate redundant TTS calls when re-generating a chapter after a voice assignment change affects only a subset of segments.

Multi-novel fixture expansion: the benchmark fixture set will be expanded beyond the current single-source-novel corpus to include chapters from multiple distinct novels, genres, and translator styles.

REFERENCES

[1] Google LLC, "Google Cloud Text-to-Speech," Google Cloud, 2024. [Online]. Available: https://cloud.google.com/text-to-speech

[2] J. Duddington, "eSpeak NG Text-to-Speech," eSpeak NG Project, 2024. [Online]. Available: https://github.com/espeak-ng/espeak-ng

[3] ElevenLabs, "ElevenLabs AI Voice Generator," ElevenLabs, Inc., 2024. [Online]. Available: https://elevenlabs.io

[4] Microsoft Corporation, "Azure AI Speech: Text to Speech," Microsoft Learn, 2024. [Online]. Available: https://learn.microsoft.com/en-us/azure/ai-services/speech-service/text-to-speech

[5] Kokoro TTS, "Kokoro: Open-Weight TTS Model," 2024. [Online]. Available: https://github.com/hexgrad/kokoro

[6] D. K. Elson and K. R. McKeown, "Automatic Attribution of Quoted Speech in Literary Narrative," in Proc. 24th AAAI Conf. Artif. Intell. (AAAI-10), Atlanta, GA, USA, 2010, pp. 1013–1019.

[7] G. Muzny, M. Fonseca, A. Chang, and D. Jurafsky, "A Two-Stage Sieve Approach for Quote Attribution," in Proc. 15th Conf. Eur. Chapter Assoc. Comput. Linguistics (EACL), Valencia, Spain, 2017, vol. 1, pp. 460–470.

[8] T. Brown et al., "Language Models are Few-Shot Learners," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 1877–1901.

[9] Google DeepMind, "Gemini 2.5 Flash," Google DeepMind, 2025. [Online]. Available: https://deepmind.google/technologies/gemini/flash/

[10] FFmpeg Project, "FFmpeg: A Complete, Cross-Platform Solution to Record, Convert and Stream Audio and Video," 2024. [Online]. Available: https://ffmpeg.org

[11] Cloudflare, Inc., "Cloudflare R2 Storage," Cloudflare, 2024. [Online]. Available: https://developers.cloudflare.com/r2

[12] S. Ramírez, "FastAPI: Modern, Fast (High-Performance) Web Framework for Building APIs with Python," 2024. [Online]. Available: https://fastapi.tiangolo.com

[13] Supabase, Inc., "Supabase: The Open Source Firebase Alternative," 2024. [Online]. Available: https://supabase.com

[14] aerkalov, "ebooklib: Python E-book Library," 2024. [Online]. Available: https://github.com/aerkalov/ebooklib